

Faculty of Sciences and Technology
University of Coimbra

Master in Informatics Engineering

Secure Software
Practical Project Report
Intermediate Submission

Authors

Daniel Pereira · 2021237092

Eduardo Marques · 2022231584

João Cardoso · 2022222301

Academic Year 2025–2026

April 3, 2026

Contents

1	System Scope and Assets	3
1.1	System Scope	3
1.2	Assets	3
1.2.1	Data Assets	4
1.2.2	System and Infrastructure Assets	4
1.2.3	Operational Assets	5
1.3	Stakeholders	6
2	Security Goals	7
2.1	Business Goal	7
2.2	Security Goals	7
3	Threat Modeling	9
4	Misuse Cases	12
4.1	MC-01 — Unauthorized Document Access	12
4.2	MC-02 — Malicious File Upload	13
4.3	MC-03 — Brute Force Authentication	13
4.4	MC-04 — Privilege Escalation via API Manipulation	14
4.5	MC-05 — Session Hijacking	15
4.6	MC-06 — SQL Injection	16
4.7	MC-07 — Cross-Site Scripting (XSS)	16
4.8	MC-08 — OS Command Injection	17
4.9	MC-09 — Resource Exhaustion via Abusive Uploads	18
4.10	MC-10 — Cross-Site Request Forgery (CSRF)	19
4.11	MC-11 — Unauthorized Branch Push / Code Injection via Repository	20
4.12	MC-12 — Secret Leakage via Pipeline	21
4.13	MC-13 — Supply Chain Attack via Compromised Dependency	22
4.14	MC-14 — Pipeline Privilege Escalation	23
5	Attack Trees	25
5.1	AT-01 — Unauthorized Document Access (IDOR)	25
5.2	AT-02 — Malicious File Upload	26
5.3	AT-03 — Brute-Force Authentication	27
5.4	AT-04 — Privilege Escalation via API Manipulation	28
5.5	Summary: Misuse Cases and Attack Trees	29
6	Risk Assessment	30

6.1	Risk Assessment Methodology	30
6.2	Risk Assessment Results	31
6.3	Discussion	34
6.3.1	Critical Risks	34
6.3.2	High Risks	35
6.3.3	Medium Risks	35
6.3.4	Low Risks	35
6.4	Threat Prioritisation Summary	35
7	Elicitation of Security Requirements	37
7.1	BON	37
7.1.1	Step 1 — Brainstorm	37
7.1.2	Step 2 — Organize	37
7.1.3	Step 3 — Name	38
7.1.4	Detailed Requirements Traceability Index	40
7.1.5	Discussion	42
8	Categorization and Prioritization of Requirements	43
8.1	Prioritize Requirements	44
8.2	Discussion	46
9	Secure Architecture and Design	47
9.1	Trust Boundaries and Security Domains	47
9.1.1	Identified Trust Boundaries	47
9.1.2	Security Domains	48
9.2	Security Enforcement Points	48
9.2.1	EP-01 — HTTPS Termination and Transport Security	48
9.2.2	EP-02 — Authentication and Session Management	48
9.2.3	EP-03 — Rate Limiting and Account Lockout	49
9.2.4	EP-04 — Role-Based Access Control (RBAC)	49
9.2.5	EP-05 — Input Validation and Injection Prevention	49
9.2.6	EP-06 — File Upload Validation and Secure Storage	50
9.2.7	EP-07 — CSRF Protection	50
9.2.8	EP-08 — Audit Logging	51
9.2.9	EP-09 — CI/CD Security Controls	51
9.2.10	EP-10 — Encryption at Rest	52
9.2.11	EP-11 — Operational Resilience and Secure Defaults	52
9.3	Architectural Overview	53
9.4	Architectural Security Principles	54
9.4.1	Least Privilege	54
9.4.2	Defense in Depth	54
9.4.3	Secure Defaults	55
9.4.4	Separation of Concerns	55
9.5	Traceability to Security Requirements	56

1. System Scope and Assets

1.1 System Scope

The system under analysis is a **web-based document management platform** designed to allow authenticated users to upload, store, and manage digital documents. The platform is accessible through a browser-based client and is backed by an application server that handles business logic, a PostgreSQL database for persistence, and a dedicated file storage system for document data.

The platform supports three distinct user roles with different levels of privilege:

- **Users**, who can authenticate, upload documents, manage their own documents, and share them with other users for review.
- **Reviewers**, who are regular users that have been granted access to documents shared with them by other users, allowing them to list and download those documents.
- **Administrators**, who manage the platform by listing registered users and enabling or disabling user accounts.

The system is containerized and implemented primarily in Python, with a frontend composed of HTML, CSS, and JavaScript. It is delivered through a CI/CD pipeline that currently supports building, testing, and packaging of the application.

1.2 Assets

An asset is anything that has value to the organization and therefore needs protection. Assets can include data, systems, applications, infrastructure, credentials, services, and reputation. The loss, disclosure, alteration, or unavailability of an asset may negatively impact the organization and its users.

The following assets have been identified within the document management platform, grouped by category.

1.2.1 Data Assets

Asset	Value / Why it needs protection	Impact if compromised
User documents	Uploaded files may contain sensitive or private information belonging to users.	Disclosure violates user privacy; loss or alteration affects data integrity.
User credentials	Passwords and usernames enable authentication; their secrecy is fundamental to access control.	Disclosure allows impersonation and unauthorized access to all user resources.
User account data	Roles and account status determine what each user is authorized to do.	Alteration can grant unauthorized privileges; loss disrupts platform operation.
Document metadata	Ownership and sharing relationships define access rights to documents.	Disclosure reveals relationships between users; alteration can break access control.
Session tokens	Tokens authenticate ongoing sessions; their secrecy is equivalent to credentials.	Disclosure enables session hijacking and full account takeover.
Audit logs	Records of system activity support accountability and incident response.	Loss or tampering with audit logs prevents detection of past attacks and removes accountability for user actions.

Table 1.1: Data assets identified in the system.

1.2.2 System and Infrastructure Assets

Asset	Value / Why it needs protection	Impact if compromised
Application server	Enforces business logic, access control, and all security mechanisms.	Compromise gives an attacker full control over the platform and its data.
Database	Stores all persistent data including credentials, account data, and document metadata.	Breach exposes all sensitive data; corruption disrupts platform operation.
Document storage	Holds the actual files uploaded by users.	Unauthorized access discloses sensitive documents; deletion causes irreversible data loss.

Asset	Value / Why it needs protection	Impact if compromised
CI/CD pipeline	Automates build, test, and delivery of the application.	Compromise can introduce malicious code into production deployments.
Application source code	Contains the implementation logic and configuration of the platform.	Disclosure exposes vulnerabilities; alteration can introduce backdoors.

Table 1.2: System and infrastructure assets identified in the system.

1.2.3 Operational Assets

Asset	Value / Why it needs protection	Impact if compromised
Platform availability	Legitimate users depend on continuous access to upload, manage, and retrieve documents.	Unavailability disrupts operations and degrades trust in the platform.
Access control policies	Define which users can access which documents and perform which actions; central to authorization.	Alteration or bypass enables unauthorized access to resources and privilege escalation.
Deployment configuration	Contains environment variables, secrets, and container settings required for secure operation.	Disclosure exposes credentials and secrets; alteration can silently weaken security controls.
Data backups	Backup copies of documents and database records are essential for recovery after incidents such as ransomware, accidental deletion, or storage failure.	Loss or compromise of backups prevents recovery after an incident; tampering with backups can render them useless or introduce corrupted data.
Source code repository	The version control repository (e.g., GitHub) is the authoritative source of application code and CI/CD configuration; it controls what gets deployed.	Unauthorized access allows introduction of malicious code or backdoors; deletion or corruption disrupts development and the ability to audit historical changes.

Table 1.3: Operational assets identified in the system.

1.3 Stakeholders

The following stakeholders interact with or have an interest in the security of the system:

- **Regular Users** — upload and manage documents; expect confidentiality and integrity of their data.
- **Reviewers** — access documents shared with them; must only see documents they are explicitly granted access to.
- **Administrators** — manage user accounts; require trusted access to sensitive operational functions.
- **Development Team** — responsible for building and maintaining the platform securely.
- **External Attackers** — may attempt to gain unauthorized access to documents, credentials, or system resources.
- **Malicious Insiders** — authenticated users who attempt to exceed their granted privileges, for instance by accessing documents they do not own.

2. Security Goals

2.1 Business Goal

The system shall provide a secure and reliable platform for users to upload, store, and share sensitive documents, enabling controlled collaboration between users and reviewers.

2.2 Security Goals

The following security goals were derived from the business goal and from the assets and stakeholders identified in the previous chapter. Each goal expresses a high-level security property that the system must satisfy.

ID	Security Goal	Related Assets
G-01	The system shall ensure that only authenticated and authorized users can access the platform and its resources.	User credentials, Session tokens, Access control policies, User account data
G-02	The system shall enforce the principle of least privilege — users operate only within their assigned role and scope.	Access control policies, User account data, Application server
G-03	The confidentiality of documents and user data shall be maintained — access is granted only to those explicitly authorized.	User documents, Document meta-data, User account data, Deployment configuration
G-04	The integrity of documents and system data shall be preserved — no unauthorized modification or deletion shall occur.	User documents, Document meta-data, Audit logs, Database, Document storage, Data backups

ID	Security Goal	Related Assets
G-05	The system shall validate and safely handle all user-supplied content, preventing malicious files or inputs from compromising the platform.	User documents, Document storage, Application server
G-06	The system shall be available and resilient against abuse scenarios, including resource exhaustion through excessive uploads or requests.	Platform availability, Application server, Document storage
G-07	The integrity of the software delivery pipeline shall be ensured, preventing the introduction of malicious or vulnerable code into the system through the CI/CD workflow.	CI/CD pipeline, Application source code, Deployment configuration, Source code repository
G-08	The system shall enforce accountability by logging security-relevant actions and supporting audit.	Audit logs, User account data, Access control policies, Data backups

Table 2.1: Security goals defined for the system.

3. Threat Modeling

Threat modeling was performed using the **STRIDE** framework, which is widely adopted in the literature and aligns well with the security goals defined for this system. Rather than applying STRIDE at a category level only, each concrete threat was identified per system component and assigned a unique identifier to enable traceability to the misuse cases and security requirements.

ID	Category	Threat Description	Component	Related MC
T-01	Spoofing	An attacker forges or steals a session token to impersonate a legitimate user.	Session management	MC-05
T-02	Spoofing	An attacker performs brute force on the login endpoint to gain unauthorized access.	Login	MC-03
T-03	Tampering	A user manipulates a document ID in an API request to access or overwrite a document they do not own.	API, Database	MC-01
T-04	Tampering	A user uploads a malicious file disguised as a document to achieve code execution on the server.	File storage, Application server	MC-02
T-05	Tampering	An attacker injects malicious SQL through user-supplied input to read, modify, or delete database records.	Database, API	MC-06
T-06	Tampering	An attacker injects malicious JavaScript into user-supplied input that is later rendered in another user's browser, enabling session theft or unauthorized actions.	Web interface, Session management	MC-07

ID	Category	Threat Description	Component	Related MC
T-07	Tampering	An attacker supplies a crafted filename or input that is passed unsanitized to an OS command, enabling arbitrary command execution on the server.	File storage, Application server	MC-08
T-08	Tampering	An attacker crafts a malicious external page that triggers state-changing requests on the platform using the victim’s active session, without their knowledge.	Session management, API endpoints	MC-10
T-09	Tampering	An attacker or malicious developer pushes code directly to a protected branch (e.g. <code>main</code>) without peer review, bypassing branch protection rules and introducing backdoors or vulnerable code into production deployments.	Source code repository, CI/CD pipeline	MC-11
T-10	Tampering	An attacker compromises a third-party library or container image used by the pipeline (supply chain attack), causing malicious or vulnerable code to be automatically pulled and deployed without detection.	CI/CD pipeline, Application source code	MC-13
T-11	Repudiation	A user performs a security-relevant action (e.g., accessing or deleting a document) that is never recorded, preventing accountability.	Audit logs	—
T-12	Information Disclosure	An authenticated user accesses documents belonging to other users due to missing server-side authorization checks.	API, File storage	MC-01

ID	Category	Threat Description	Component	Related MC
T-13	Information Disclosure	Sensitive data (credentials, tokens, document content) is exposed through unencrypted communication or insecure storage.	Network, Session management	MC-05
T-14	Information Disclosure	Secrets, API keys, or database credentials are hardcoded in source code, CI/CD configuration files, or pipeline logs, exposing them to anyone with read access to the repository or the pipeline execution environment.	CI/CD pipeline, Deployment configuration	MC-12
T-15	Denial of Service	An attacker exhausts server or storage resources through repeated or oversized file uploads.	Upload functionality, File storage	MC-09
T-16	Elevation of Privilege	A regular user manipulates API requests to access administrator-only endpoints and perform privileged actions.	API, Access control	MC-04
T-17	Elevation of Privilege	A user gains access to documents shared with another reviewer by manipulating sharing parameters.	API, Database	MC-01
T-18	Elevation of Privilege	A developer or attacker exploits misconfigured pipeline permissions or overly broad CI/CD job credentials to trigger unauthorised deployments to the production environment or to extract production secrets from the pipeline execution context.	CI/CD pipeline, Deployment configuration	MC-14

Table 3.1: STRIDE threat analysis.

4. Misuse Cases

The following misuse cases describe scenarios in which the system could be abused by an attacker or a malicious insider. Each misuse case is structured following the SQUARE methodology, identifying the mis-actor, the abuse scenario, and the desired prevention, detection, and recovery guarantees.

4.1 MC-01 — Unauthorized Document Access

Number:	MC-01
Name:	Unauthorized document access
Scope:	Document access control
Priority:	_ Low _ Medium <u>x</u> High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	<u>x</u> Confidentiality <u>x</u> Integrity _ Availability
Mis-actor:	Authenticated user (no access rights to the target document)
Description:	An authenticated user attempts to access or download a document they do not own and that has not been shared with them, by manipulating request parameters (e.g., document ID in URL).
Pre-conditions:	User is authenticated. Document exists and belongs to another user. No server-side authorization check is enforced on document access.
Post-conditions:	Worst Case Threat: The user successfully downloads a sensitive document belonging to another user. The access is never detected.
	Wanted Prevention Guarantee: Server-side authorization check on every document access request, verifying ownership or explicit share permission.
	Wanted Detection Guarantee: All document access attempts are logged with user ID, document ID, and timestamp. Unauthorized attempts trigger an alert.
	Wanted Recovery Guarantee: Revoke access, invalidate session, notify document owner, and audit all accesses made by the mis-actor.
Related STRIDE:	Information Disclosure, Elevation of Privilege

Related Goals:	G-01, G-02, G-03
-----------------------	------------------

Table 4.1: Misuse case MC-01.

4.2 MC-02 — Malicious File Upload

Number:	MC-02
Name:	Malicious file upload
Scope:	Document upload functionality
Priority:	_ Low _ Medium <u>x</u> High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	_ Confidentiality <u>x</u> Integrity <u>x</u> Availability
Mis-actor:	Authenticated user
Description:	An authenticated user uploads a file containing malicious content (e.g., a web shell, executable, or script) disguised as a legitimate document, attempting to achieve code execution on the server.
Pre-conditions:	User is authenticated. Upload functionality is available. No file type or content validation is enforced.
Post-conditions:	Worst Case Threat: Malicious file is stored and later executed, giving the attacker full control over the server.
	Wanted Prevention Guarantee: File type validation, MIME type inspection, and sandboxed storage to prevent execution of uploaded files.
	Wanted Detection Guarantee: All uploads are logged with file metadata. Suspicious file types trigger an alert.
	Wanted Recovery Guarantee: Remove malicious file, audit all uploads from the mis-actor, restore system to a clean state.
Related STRIDE:	Tampering, Elevation of Privilege
Related Goals:	G-04, G-05

Table 4.2: Misuse case MC-02.

4.3 MC-03 — Brute Force Authentication

Number:	MC-03
Name:	Brute force authentication
Scope:	Authentication
Priority:	_ Low _ Medium <u>x</u> High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet

Security Attributes Affected:	<u>x</u> Confidentiality _ Integrity _ Availability
Mis-actor:	External attacker (unauthenticated)
Description:	An attacker attempts to gain access to the platform by systematically trying username/password combinations against the login endpoint.
Pre-conditions:	Login endpoint is publicly accessible. No rate limiting or account lockout is in place.
Post-conditions:	Worst Case Threat: Attacker successfully authenticates as a legitimate user or administrator. Actions are never detected.
	Wanted Prevention Guarantee: Rate limiting on login endpoint, account lockout after repeated failures, and strong password policy enforcement.
	Wanted Detection Guarantee: Failed login attempts are logged with IP and timestamp. Repeated failures from the same source trigger an alert.
	Wanted Recovery Guarantee: Lock compromised account, force password reset, notify the user, and audit all actions performed during the unauthorized session.
Related STRIDE:	Spoofing
Related Goals:	G-01, G-08

Table 4.3: Misuse case MC-03.

4.4 MC-04 — Privilege Escalation via API Manipulation

Number:	MC-04
Name:	Privilege escalation via API manipulation
Scope:	Authorization, API endpoints
Priority:	_ Low _ Medium <u>x</u> High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	<u>x</u> Confidentiality <u>x</u> Integrity _ Availability
Mis-actor:	Authenticated user (regular user or reviewer)
Description:	A regular user manipulates API requests (e.g., modifying role parameters or directly accessing admin endpoints) to perform actions reserved for administrators, such as enabling or disabling user accounts.
Pre-conditions:	User is authenticated. Server does not enforce role-based checks server-side on privileged endpoints.
Post-conditions:	Worst Case Threat: User gains admin-level control over the platform, disabling accounts or accessing all documents. Actions are never detected.

	Wanted Prevention Guarantee: Server-side role-based access control enforced on every endpoint, regardless of client-side restrictions. Wanted Detection Guarantee: All access attempts to privileged endpoints are logged with user ID, role, and timestamp. Unauthorized attempts trigger an alert. Wanted Recovery Guarantee: Revoke session, restore any changes made, audit all actions performed by the mis-actor.
Related STRIDE:	Elevation of Privilege, Tampering
Related Goals:	G-01, G-02

Table 4.4: Misuse case MC-04.

4.5 MC-05 — Session Hijacking

Number:	MC-05
Name:	Session hijacking
Scope:	Session management
Priority:	_ Low <u>x</u> Medium _ High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	<u>x</u> Confidentiality <u>x</u> Integrity _ Availability
Mis-actor:	External attacker
Description:	An attacker steals a valid session token (e.g., via XSS, network sniffing, or insecure cookie storage) and uses it to impersonate a legitimate user without their knowledge.
Pre-conditions:	Session tokens are transmitted over HTTP or stored without Secure/HttpOnly flags. Application is vulnerable to XSS or token leakage.
Post-conditions:	Worst Case Threat: Attacker fully impersonates a legitimate user or admin for the duration of the session. Actions are attributed to the victim.
	Wanted Prevention Guarantee: HTTPS enforcement, Secure and HttpOnly cookie flags, SameSite policy, and short session expiry. XSS prevention via input sanitization and CSP headers.
	Wanted Detection Guarantee: Detect concurrent sessions from different IPs or user agents for the same token. Log all session creation and termination events.
	Wanted Recovery Guarantee: Invalidate all active sessions for the affected user, force re-authentication, and audit all actions performed during the hijacked session.
Related STRIDE:	Spoofing, Information Disclosure
Related Goals:	G-01, G-03, G-08

Table 4.5: Misuse case MC-05.

4.6 MC-06 — SQL Injection

Number:	MC-06
Name:	SQL injection
Scope:	Database, API input handling
Priority:	☐ Low ☐ Medium ☒ High
Deployment Environment:	☐ Intranet ☒ Extranet/Internet
Security Attributes Affected:	☒ Confidentiality ☒ Integrity ☒ Availability
Mis-actor:	External attacker or authenticated user
Description:	An attacker injects malicious SQL code through user-supplied input fields (e.g., login form, search parameters) that is incorporated into database queries without proper sanitization, allowing them to read, modify, or delete database records.
Pre-conditions:	User input is incorporated into SQL queries without parameterization or sanitization. The application does not use an ORM or prepared statements consistently.
Post-conditions:	Worst Case Threat: Attacker dumps the entire database, including credentials and all document metadata, or deletes all records. Authentication may be bypassed entirely.
	Wanted Prevention Guarantee: Exclusive use of parameterized queries or ORM abstractions. Input validation on all user-supplied data. Principle of least privilege applied to database user accounts.
	Wanted Detection Guarantee: Anomalous or malformed query patterns are logged and trigger an alert. Database error messages are never exposed to the client.
	Wanted Recovery Guarantee: Restore database from backup, invalidate all active sessions, audit all queries executed during the attack window, and force password reset for all users.
Related STRIDE:	Tampering, Information Disclosure, Elevation of Privilege
Related Goals:	G-01, G-03, G-04, G-05

Table 4.6: Misuse case MC-06.

4.7 MC-07 — Cross-Site Scripting (XSS)

Number:	MC-07
Name:	Cross-site scripting (XSS)
Scope:	Web interface, input handling
Priority:	☐ Low ☐ Medium ☒ High
Deployment Environment:	☐ Intranet ☒ Extranet/Internet

Security Attributes Affected:	<u>x</u> Confidentiality <u>x</u> Integrity _ Availability
Mis-actor:	Authenticated user
Description:	An attacker injects malicious JavaScript into a user-supplied input field (e.g., document name or other displayed field) that is later rendered unsanitized in another user’s browser, allowing the attacker to steal session tokens, perform actions on behalf of the victim, or redirect them to a malicious page.
Pre-conditions:	User input is reflected or stored and rendered in the browser without proper sanitization or encoding. No Content Security Policy (CSP) is enforced.
Post-conditions:	Worst Case Threat: Attacker steals the session token of an administrator and gains full control over the platform. The attack is never detected.
	Wanted Prevention Guarantee: All user-supplied input is sanitized and HTML-encoded before being rendered. A strict Content Security Policy header is enforced on all responses.
	Wanted Detection Guarantee: Anomalous script patterns in input fields are logged. CSP violation reports are collected and reviewed.
	Wanted Recovery Guarantee: Invalidate all active sessions, sanitize or remove affected content, and audit all actions performed during the attack.
Related STRIDE:	Tampering, Information Disclosure, Spoofing
Related Goals:	G-01, G-03, G-05

Table 4.7: Misuse case MC-07.

4.8 MC-08 — OS Command Injection

Number:	MC-08
Name:	OS command injection
Scope:	File upload, server-side input handling
Priority:	_ Low _ Medium <u>x</u> High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	<u>x</u> Confidentiality <u>x</u> Integrity <u>x</u> Availability
Mis-actor:	Authenticated user
Description:	An attacker supplies a crafted filename or other user-controlled input that is passed unsanitized to an OS-level command (e.g., via Python’s <code>os.system()</code> or <code>subprocess</code>) on the server, enabling arbitrary command execution on the underlying operating system.

Pre-conditions:	User-supplied input (e.g., filename) is incorporated into OS commands without sanitization. The application invokes shell commands as part of file processing.
Post-conditions:	Worst Case Threat: Attacker executes arbitrary commands on the server, exfiltrates all data, or destroys the system. Full server compromise is achieved.
	Wanted Prevention Guarantee: Never pass user-supplied input to OS commands. Use safe Python APIs for file handling. If shell commands are necessary, use parameterized subprocess calls with shell=False and strict input validation.
	Wanted Detection Guarantee: All file processing operations are logged with the original filename and user ID. Anomalous characters in filenames trigger an alert.
	Wanted Recovery Guarantee: Isolate the compromised container, restore from a clean state, audit all commands executed, and invalidate all active sessions.
Related STRIDE:	Tampering, Elevation of Privilege
Related Goals:	G-04, G-05, G-08

Table 4.8: Misuse case MC-08.

4.9 MC-09 — Resource Exhaustion via Abusive Uploads

Number:	MC-09
Name:	Resource exhaustion via abusive uploads
Scope:	Upload functionality, platform availability
Priority:	_ Low <u>x</u> Medium _ High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	_ Confidentiality _ Integrity <u>x</u> Availability
Mis-actor:	Authenticated user
Description:	An authenticated user repeatedly uploads large files or sends a high volume of requests to the upload endpoint, exhausting server storage or processing capacity and rendering the platform unavailable to other users.
Pre-conditions:	User is authenticated. No upload rate limiting, file size limit, or storage quota is enforced.
Post-conditions:	Worst Case Threat: Storage is completely exhausted, the application server becomes unresponsive, and the platform is unavailable to all users. The attack is never detected.
	Wanted Prevention Guarantee: Enforce per-user storage quotas, maximum file size limits, and rate limiting on the upload endpoint.

	Wanted Detection Guarantee: Monitor storage usage and upload rates. Trigger alerts when thresholds are exceeded or when a single user accounts for an anomalous volume of uploads. Wanted Recovery Guarantee: Remove abusive uploads, suspend the offending account, restore service availability, and audit all upload activity from the mis-actor.
Related STRIDE:	Denial of Service
Related Goals:	G-06

Table 4.9: Misuse case MC-09.

4.10 MC-10 — Cross-Site Request Forgery (CSRF)

Number:	MC-10
Name:	Cross-site request forgery (CSRF)
Scope:	Session management, state-changing endpoints
Priority:	_ Low <u>x</u> Medium _ High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	_ Confidentiality <u>x</u> Integrity _ Availability
Mis-actor:	External attacker
Description:	An attacker crafts a malicious external page that silently sends a state-changing request (e.g., delete a document, change sharing settings) to the platform on behalf of an authenticated user who visits that page. The server accepts the request because it originates from a valid session, without verifying that the request was intentionally initiated by the user.
Pre-conditions:	The user is authenticated and has an active session. The server does not validate the origin of requests (no CSRF token, no SameSite cookie policy).
Post-conditions:	Worst Case Threat: An attacker silently deletes or shares sensitive documents belonging to the victim, or modifies account settings, without the victim's knowledge.
	Wanted Prevention Guarantee: Enforce CSRF tokens on all state-changing requests. Set SameSite=Strict or SameSite=Lax on session cookies. Verify the Origin and Referer headers on sensitive endpoints.
	Wanted Detection Guarantee: Log all state-changing requests with origin information. Flag requests missing expected CSRF tokens or with unexpected referrer headers.
	Wanted Recovery Guarantee: Revert unauthorized changes where possible, notify the affected user, invalidate the session, and audit all actions performed during the attack window.

Related STRIDE:	Tampering, Spoofing
Related Goals:	G-04, G-08

Table 4.10: Misuse case MC-10.

4.11 MC-11 — Unauthorized Branch Push / Code Injection via Repository

Number:	MC-11
Name:	Unauthorized branch push / code injection via repository
Scope:	Source code repository, CI/CD pipeline
Priority:	_ Low _ Medium <u>x</u> High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	_ Confidentiality <u>x</u> Integrity _ Availability
Mis-actor:	Malicious insider (developer with repository access) or external attacker with stolen credentials
Description:	An attacker with write access to the repository pushes malicious code directly to a protected branch (e.g. <code>main</code> or <code>production</code>), bypassing the mandatory peer-review process. The CI/CD pipeline automatically builds and deploys the tampered code without any human inspection, introducing a backdoor or vulnerability into the running application.
Pre-conditions:	Branch protection rules are absent or misconfigured. The attacker has obtained valid repository credentials (own account or stolen). The CI/CD pipeline deploys automatically on push to the protected branch.
Post-conditions:	Worst Case Threat: Malicious code is deployed to production undetected, creating a persistent backdoor that allows the attacker to exfiltrate data or maintain long-term access to the system.
	Wanted Prevention Guarantee: Enforce branch protection rules on all production branches, requiring at least one approved code review before merging. Disable direct pushes for all identities, including repository owners. Enforce signed commits where feasible.
	Wanted Detection Guarantee: Log all push and merge events with author identity and timestamp. Alert on any direct push to a protected branch or any merge without a required review approval.

	Wanted Recovery Guarantee: Revert the malicious commit, redeploy from the last known-good state, rotate all credentials potentially exposed by the compromised code, and audit all pipeline executions triggered since the tampered push.
Related STRIDE:	Tampering
Related Goals:	G-04, G-07

Table 4.11: Misuse case MC-11.

4.12 MC-12 — Secret Leakage via Pipeline

Number:	MC-12
Name:	Secret leakage via pipeline
Scope:	CI/CD pipeline, deployment configuration
Priority:	_ Low <u>x</u> Medium _ High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	<u>x</u> Confidentiality _ Integrity _ Availability
Mis-actor:	External attacker or malicious insider with read access to the repository or pipeline logs
Description:	A developer hardcodes secrets (API keys, database credentials, cryptographic keys) directly in source code or CI/CD configuration files, or a pipeline job prints them to its execution log. Any person with read access to the repository or to the CI/CD log output can retrieve those secrets and use them to access protected resources, impersonate services, or compromise the database.
Pre-conditions:	Secrets are present in plaintext in the repository or in pipeline log output. No secret-scanning step is integrated into the pipeline. Repository or pipeline logs are accessible to the attacker (public repository or stolen credentials).
Post-conditions:	Worst Case Threat: The attacker retrieves a production database password or API key, gains direct access to the database, and silently exfiltrates all user data and documents. The exposure is never detected until a secondary incident occurs.
	Wanted Prevention Guarantee: Prohibit hardcoded secrets in source code and CI/CD files. Inject all secrets via a dedicated secret management mechanism (e.g., environment secrets, a vault). Integrate automated secret-detection scanning as a mandatory blocking stage in the pipeline.

	Wanted Detection Guarantee: Run secret-scanning tools on every commit and pull request. Alert immediately if a pattern matching a credential or key is detected in any file or pipeline log. Wanted Recovery Guarantee: Immediately rotate all exposed secrets, audit all access made using the leaked credentials, remove the secret from version-control history using a history rewrite, and force re-authentication for all active sessions.
Related STRIDE:	Information Disclosure
Related Goals:	G-03, G-07

Table 4.12: Misuse case MC-12.

4.13 MC-13 — Supply Chain Attack via Compromised Dependency

Number:	MC-13
Name:	Supply chain attack via compromised dependency
Scope:	CI/CD pipeline, application dependencies
Priority:	☐ Low ☐ Medium ☒ High
Deployment Environment:	☐ Intranet ☒ Extranet/Internet
Security Attributes Affected:	☒ Confidentiality ☒ Integrity ☒ Availability
Mis-actor:	External attacker (third-party maintainer or package registry compromise)
Description:	An attacker compromises a third-party library or base container image that the pipeline pulls automatically during the build stage. Because the pipeline uses unpinned dependency versions or does not verify image digests, the malicious version is silently pulled and built into the application image. The compromised artefact is then deployed to production without any additional validation, giving the attacker code-execution capabilities inside the running container.
Pre-conditions:	Dependencies are specified without pinned versions or cryptographic digests. The pipeline does not run dependency vulnerability scanning or image verification. The attacker has succeeded in publishing a malicious version of a dependency used by the project.
Post-conditions:	Worst Case Threat: The compromised dependency executes arbitrary code inside the production container, exfiltrating secrets and user data to an attacker-controlled server. The attack is invisible to standard application-level monitoring.

	Wanted Prevention Guarantee: Pin all dependencies to specific versions and verify cryptographic hashes. Pull base container images only from trusted registries and verify image digests before use. Integrate automated dependency vulnerability scanning and Software Composition Analysis (SCA) as mandatory blocking pipeline stages. Wanted Detection Guarantee: Alert on any dependency version change introduced without explicit developer action. Run SCA tools on every build and report newly discovered CVEs in used packages. Wanted Recovery Guarantee: Roll back to the last known-good build, remove the compromised image from the registry, rotate all secrets that may have been exposed, and audit all network connections made by the affected container during its runtime.
Related STRIDE:	Tampering
Related Goals:	G-04, G-05, G-07

Table 4.13: Misuse case MC-13.

4.14 MC-14 — Pipeline Privilege Escalation

Number:	MC-14
Name:	Pipeline privilege escalation
Scope:	CI/CD pipeline permissions, deployment configuration
Priority:	_ Low <u>x</u> Medium _ High
Deployment Environment:	_ Intranet <u>x</u> Extranet/Internet
Security Attributes Affected:	<u>x</u> Confidentiality <u>x</u> Integrity _ Availability
Mis-actor:	Malicious insider (developer) or external attacker who has gained repository write access
Description:	A developer or attacker modifies a CI/CD workflow definition to abuse the pipeline’s overly broad execution permissions. Because CI/CD jobs run with credentials that grant access to production infrastructure or secret stores, the attacker crafts a workflow step that reads production secrets from the secret store, triggers an unauthorised deployment to the production environment, or exfiltrates environment variables containing sensitive credentials to an external endpoint.
Pre-conditions:	CI/CD job tokens or service accounts have broader permissions than required (e.g., write access to production, unrestricted secret access). Workflow definition files can be modified by anyone with repository write access without additional approval. No separation exists between pipeline environments (e.g., staging vs. production).

Post-conditions:	Worst Case Threat: The attacker deploys an arbitrary image to the production environment or extracts all production secrets via a rogue pipeline step, achieving persistent access equivalent to a full infrastructure compromise.
	Wanted Prevention Guarantee: Apply the principle of least privilege to all CI/CD job tokens and service accounts. Restrict production deployments to dedicated protected workflows that require manual approval. Separate pipeline credentials per environment and scope secret access to only the jobs that strictly require it.
	Wanted Detection Guarantee: Log all pipeline executions with the triggering identity, workflow file hash, and any secrets accessed. Alert on any workflow modification and on any pipeline job that attempts to access secrets outside its defined scope.
	Wanted Recovery Guarantee: Revoke all pipeline tokens and rotate all secrets that were in scope of the compromised job. Roll back any unauthorised deployment, audit all pipeline runs executed since the workflow was modified, and re-establish least-privilege permissions across all CI/CD service accounts.
Related STRIDE:	Elevation of Privilege
Related Goals:	G-02, G-07, G-08

Table 4.14: Misuse case MC-14.

5. Attack Trees

Attack trees provide a structured and hierarchical representation of how an attacker may achieve a given malicious goal. Each tree is rooted at a high-level attack objective (the *root node*), which is progressively decomposed into sub-goals and, ultimately, into leaf nodes that represent concrete, atomic attack steps. Internal nodes are combined using **OR** (any child suffices to achieve the parent) or **AND** (all children must be achieved simultaneously).

Four attack trees were selected for the first four identified misuse cases. Each tree is presented as a diagram followed by a description of the leaf nodes and their traceability to threats and security requirements.

5.1 AT-01 — Unauthorized Document Access (IDOR)

Root goal: Access a document owned by another user without authorization.

Related misuse case: MC-01

Related threats: T-03, T-12

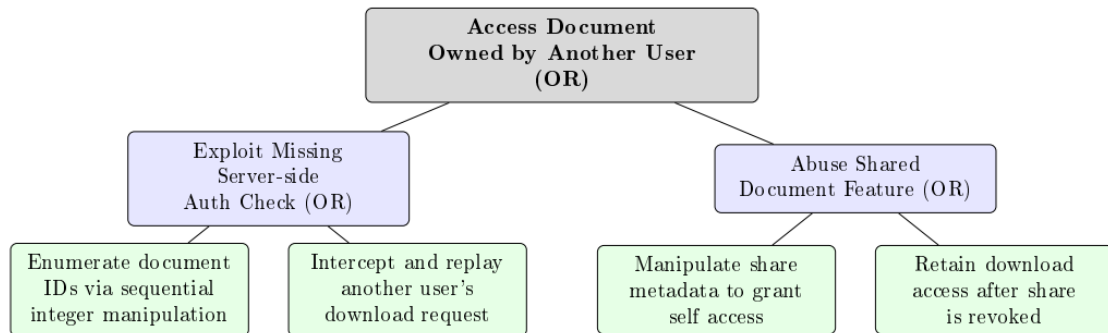


Figure 5.1: AT-01 — Unauthorized document access (IDOR).

Leaf node descriptions.

- **A1 — Sequential ID enumeration.** The application exposes document resources via predictable integer identifiers in the URL or API path (e.g., /documents/42). An attacker iterates over numeric IDs to discover and download documents belonging to other accounts. This attack succeeds if no server-side ownership check is performed before serving the file.

- **A2 — Request replay.** The attacker captures a legitimate document download request (e.g., via a shared link or network interception) and replays it from a different session, bypassing any client-side restriction.
- **B1 — Share metadata manipulation.** The attacker crafts an API request that directly modifies the sharing relationship in the database (e.g., by submitting a forged `shared_with` field), granting themselves reviewer access to a document they do not own.
- **B2 — Stale access after revocation.** Once sharing is revoked by the document owner, the server continues to honour previously cached or bookmarked download URLs because it does not re-validate the sharing relationship at access time.

Traceability. Threats mitigated: T-03, T-12. Requirements addressed: SR-03, SR-04, SR-06, SR-11. Enforcement point: EP-04 (RBAC and ownership checks).

5.2 AT-02 — Malicious File Upload

Root goal: Execute arbitrary code on the server via a crafted upload.

Related misuse case: MC-02

Related threats: T-04, T-07

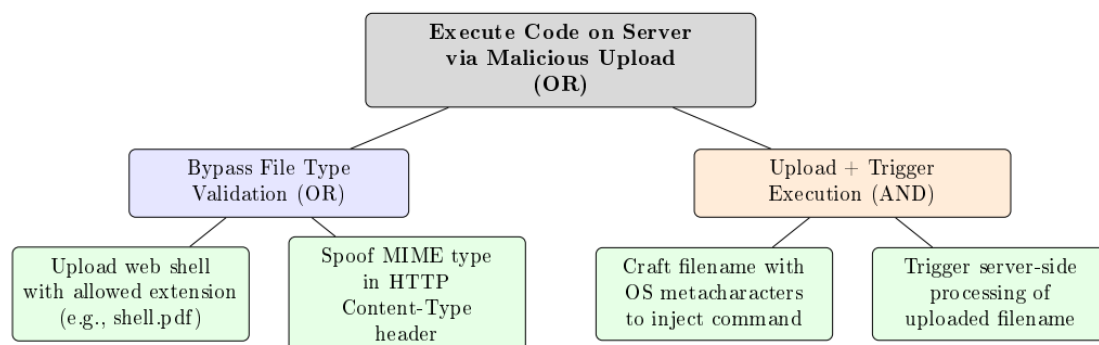


Figure 5.2: AT-02 — Malicious file upload leading to code execution.

Leaf node descriptions.

- **A1 — Extension spoofing.** The attacker renames a web shell (e.g., a Python or PHP script) with an extension accepted by the application (e.g., `.pdf` or `.txt`). If the server relies solely on the file extension and not on actual content inspection, the malicious file is stored and potentially accessible for later execution or delivery.
- **A2 — MIME type spoofing.** The attacker manipulates the `Content-Type` header in the upload request to declare a benign MIME type (e.g., `application/pdf`) while sending executable content. If the server trusts the declared MIME type without independently verifying the file’s magic bytes, the malicious content passes validation.

- **B1 — Filename command injection.** The attacker supplies a filename containing OS shell metacharacters (e.g., `report; rm -rf /`) that, when passed unsanitized to an OS-level function (e.g., `os.system`, `subprocess`), causes the server to execute the embedded command.
- **B2 — Trigger via processing.** The crafted filename alone is not sufficient; the attacker also needs to trigger a server-side operation that processes the filename (e.g., a thumbnail generation step, a virus scan invocation, or a log rotation script). Both B1 and B2 must succeed simultaneously (AND gate).

Traceability. Threats mitigated: T-04, T-07. Requirements addressed: SR-07, SR-18, SR-23, SR-24. Enforcement point: EP-06 (file type validation, magic byte inspection, sandboxed storage).

5.3 AT-03 — Brute-Force Authentication

Root goal: Gain authenticated access without knowing valid credentials.

Related misuse case: MC-03

Related threats: T-02

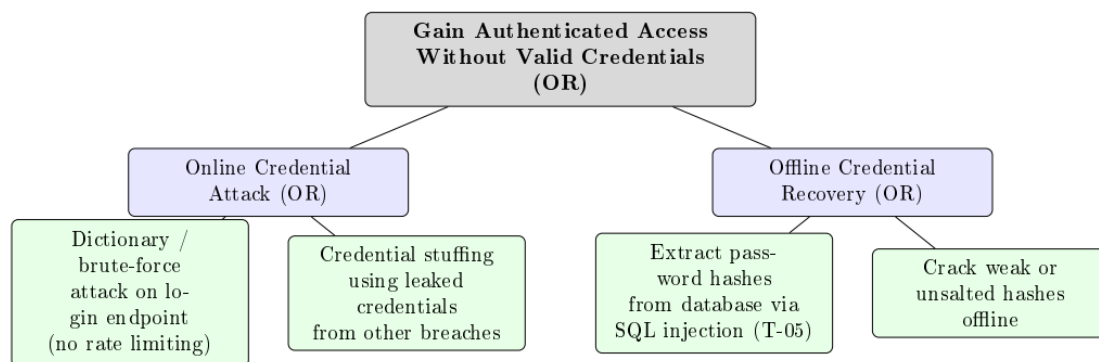


Figure 5.3: AT-03 — Brute-force authentication.

Leaf node descriptions.

- **A1 — Online brute force.** The attacker submits a high volume of login requests, systematically iterating over password candidates from a dictionary or exhaustive character space. This attack is practical only if the endpoint is not rate-limited and the application does not lock accounts after a defined number of consecutive failures.
- **A2 — Credential stuffing.** The attacker uses username/password pairs leaked from third-party breaches. Because users frequently reuse passwords across services, a significant fraction of valid credentials may be found in publicly available breach datasets without any brute-force effort.
- **B1 — Hash extraction via SQL injection.** As a prerequisite to offline cracking, the attacker first extracts password hashes from the database using SQL injection (T-05).

- **B2 — Offline hash cracking.** Using extracted hashes, the attacker performs offline password cracking using GPU-accelerated tools (e.g., hashcat). Weak hashing algorithms (e.g., MD5, SHA-1 without a salt) or the absence of a strong key derivation function (e.g., bcrypt, Argon2) make this feasible in practice.

Traceability. Threats mitigated: T-02. Requirements addressed: SR-01, SR-10, SR-13, SR-22, SR-23. Enforcement points: EP-02 (authentication), EP-03 (rate limiting and account lockout).

5.4 AT-04 — Privilege Escalation via API Manipulation

Root goal: Gain administrator-level capabilities as a regular user.

Related misuse case: MC-04

Related threats: T-16

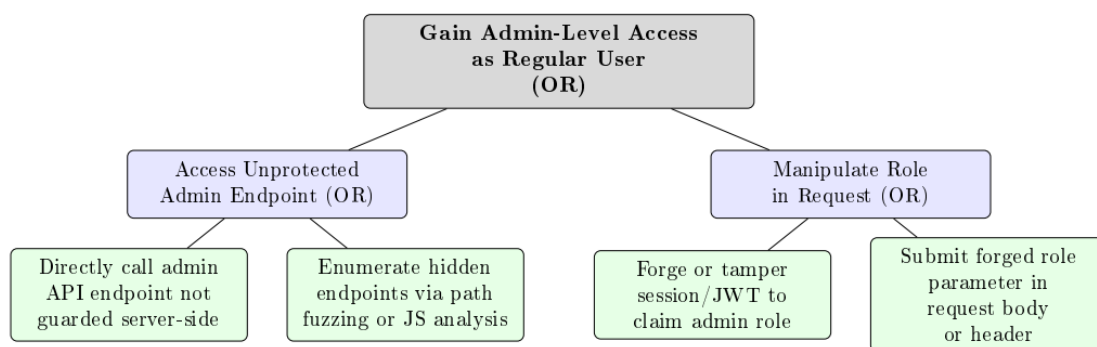


Figure 5.4: AT-04 — Privilege escalation via API manipulation.

Leaf node descriptions.

- **A1 — Unguarded admin endpoint.** The application exposes administrative endpoints (e.g., `/admin/users` or `/admin/disable`) that are only hidden from the UI but are not protected by a server-side role check. Any authenticated user can invoke them directly by crafting an HTTP request, bypassing the client-side navigation restriction.
- **A2 — Endpoint enumeration.** The attacker discovers undocumented or hidden administrative endpoints by fuzzing URL paths (using tools such as `ffuf` or `gobuster`) or by statically analyzing JavaScript bundles that may contain route definitions.
- **B1 — Session or JWT forgery.** If session tokens are signed with a weak or leaked secret, or if the application uses JWTs without robust validation, the attacker can craft a token that asserts an elevated role (e.g., `"role": "admin"`) and present it to gain access to privileged functionality.
- **B2 — Role parameter injection.** The application trusts a role or permission value submitted by the client in the request body or as a query parameter,

rather than deriving the role exclusively from the server-side session. The attacker adds or modifies this parameter to claim administrative privileges.

Traceability. Threats mitigated: T-16. Requirements addressed: SR-03, SR-04, SR-15, SR-16, SR-19, SR-27, SR-28. Enforcement point: EP-04 (RBAC middleware, role guards on every endpoint).

5.5 Summary: Misuse Cases and Attack Trees

Table 5.1 summarises the mapping between misuse cases, the corresponding attack trees, and the threats they address.

AT ID	Root Goal	Misuse Case	Threats
AT-01	Access document owned by another user	MC-01	T-03, T-12
AT-02	Execute code on server via malicious upload	MC-02	T-04, T-07
AT-03	Gain authenticated access without valid credentials	MC-03	T-02
AT-04	Gain admin-level access as a regular user	MC-04	T-16

Table 5.1: Summary of attack trees, related misuse cases and threats.

6. Risk Assessment

Risk assessment was conducted following the **NIST SP 800-30** model, which was identified as the most suitable methodology for projects of this nature in the SQUARE literature review. The process maps each threat identified in Chapter 3 to a **likelihood** value and an **impact** value, derives a **risk level** from their combination, and produces a prioritised set of results to feed into the security requirements elicitation stage (Step 6) and requirements prioritisation stage (Step 8).

6.1 Risk Assessment Methodology

Each threat was evaluated along two independent dimensions:

- **Likelihood** (*Low / Medium / High*) — the probability that the threat is successfully exploited, considering the system’s exposure (public web application), the technical difficulty of the attack, and the presence or absence of existing controls.
- **Impact** (*Low / Medium / High*) — the potential consequence of a successful attack on the confidentiality, integrity, and availability of the assets identified in Chapter 1.

The **risk level** is derived from the 3×3 matrix shown in Figure 6.1, following the NIST SP 800-30 risk determination methodology.

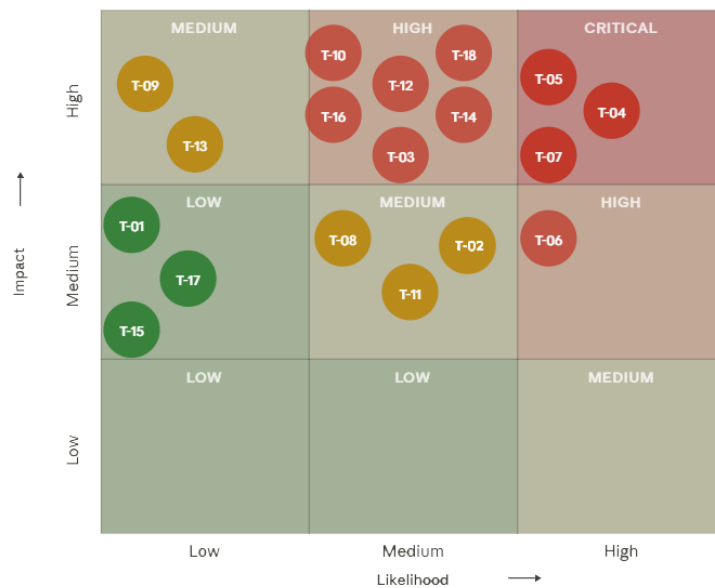


Figure 6.1: Risk matrix — STRIDE threats.

6.2 Risk Assessment Results

Table 6.1 presents the complete risk assessment for all threats identified via STRIDE analysis in Chapter 3. Each entry includes the likelihood and impact ratings, the derived risk level, and a brief justification.

ID	Threat	Like- lihood	Impact	Risk	Justification
T-05	SQL Injection	High	High	Critical	Direct incorporation of user input into database queries; successful exploitation exposes all credentials, metadata, and document records. Authentication bypass is also possible.
T-04	Malicious File Upload	High	High	Critical	No file type or content validation is enforced; an attacker can upload a web shell or executable and achieve remote code execution on the server.
T-07	OS Command Injection	High	High	Critical	User-controlled input (e.g., filename) passed unsanitised to OS-level commands; exploitation is trivial and leads to full server compromise.
T-12	Unauthorised Document Access (IDOR)	Medium	High	High	Manipulation of document IDs in API requests; missing server-side authorisation checks are a common vulnerability in document management systems and are silent by default.
T-16	Privilege Escalation via API	Medium	High	High	Admin-only endpoints reachable by regular users if role-based checks are absent server-side; allows disabling accounts, accessing all documents, or modifying platform configuration.

ID	Threat		Like- lihood	Impact	Risk	Justification
T-06	Cross-Site Scripting (XSS)		High	Medium	High	Input fields (e.g., document name) rendered unsanitised in the browser; widely exploited via automated scanners and easily escalated to session hijacking.
T-03	IDOR — Tampering		Medium	High	High	Focus is on unauthorised modification or overwrite of documents rather than disclosure. Partially overlaps with T-12; requires authenticated access.
T-10	Supply Chain Attack via Compromised Dependency		Medium	High	High	Pipeline pulls dependencies without pinned versions or digest verification; a compromised third-party library or base image is silently built into the production artefact and can achieve code execution inside the running container. Detection is particularly difficult as the malicious code is indistinguishable from legitimate dependency code.
T-18	Pipeline Privilege Escalation		Medium	High	High	CI/CD job tokens or service accounts may carry broader permissions than required; a malicious developer or attacker with repository write access can craft a workflow step to extract production secrets or trigger unauthorised deployments, with potentially catastrophic impact on the confidentiality and integrity of the system.

ID	Threat		Like- lihood	Impact	Risk	Justification
T-14	Secret Leakage via Pipeline		Medium	High	High	Secrets hardcoded in source files or printed to pipeline logs are accessible to anyone with read access to the repository or CI/CD execution environment; leaked database credentials or API keys can grant direct, persistent access to production data with no further exploitation required.
T-02	Brute Force Authentication	Au-	Medium	Medium	Medium	Public login endpoint without rate limiting or account lockout; automated credential-stuffing tools are freely available and routinely used against web applications.
T-08	Cross-Site Request Forgery (CSRF)		Medium	Medium	Medium	State-changing requests (delete, share) forgeable if no CSRF token is validated; requires the victim to visit an attacker-controlled page.
T-13	Sensitive Data Exposure	Ex-	Low	High	Medium	Residual risk if HTTPS is not enforced or session tokens are stored without Secure/HttpOnly flags; impact is high but likelihood is controlled by standard transport-layer controls.
T-11	Repudiation / Missing Audit Logs	/ Audit	Medium	Medium	Medium	Does not cause direct data loss but undermines detection and incident response; addressed through the audit and accountability goal G-08.

ID	Threat	Like- lihood	Impact	Risk	Justification
T-09	Unauthorised Branch Push / Code Injection	Low	High	Medium	Requires the attacker to have valid repository write credentials and that branch protection rules are absent or misconfigured; impact is high if malicious code reaches production, but likelihood is controlled by access management and branch protection enforcement.
T-01	Session Hijacking	Low	Medium	Low	Requires a precondition (XSS or active network sniffing); mitigated by HTTPS enforcement and correct cookie flags (Secure, HttpOnly, SameSite).
T-17	Reviewer Sharing Bypass	Low	Medium	Low	Manipulation of sharing parameters to access documents shared with another reviewer; narrow attack surface requiring prior knowledge of sharing relationships.
T-15	Resource Exhaustion via Abusive Uploads	Low	Medium	Low	Requires an authenticated account; impact is bounded by per-user storage quotas and rate limiting when these controls are in place.

Table 6.1: Risk assessment results.

6.3 Discussion

6.3.1 Critical Risks

Threats T-05, T-04, and T-07 form an **injection cluster** that shares a common root cause: user-supplied input is incorporated into sensitive operations (database queries, file storage, OS commands) without proper validation or sanitisation. All three carry a *High* likelihood because exploitation requires no special privilege — an authenticated user is sufficient — and the underlying vulnerability patterns are well-known and widely scanned for. The potential impact is catastrophic in all cases, ranging from full database compromise to remote code execution and complete server takeover. These threats must be addressed as the highest priority in the security

requirements.

6.3.2 High Risks

T-12 and T-16 represent **broken access control**, the most prevalent web application vulnerability class according to the OWASP Top 10. Both threats are particularly dangerous because they are silent by default: without comprehensive server-side logging and authorisation checks, unauthorised access can go undetected indefinitely. T-03 shares the same root cause, focusing on unauthorised modification or overwrite of documents rather than disclosure. T-06 (XSS) carries high likelihood due to the availability of automated attack tooling and can escalate to full session hijacking if exploited. T-10 (Supply Chain Attack) and T-14 (Secret Leakage via Pipeline) extend the High tier to the CI/CD domain: a compromised dependency or an exposed secret can grant an attacker code-execution or direct database access without any application-layer exploitation, and both are difficult to detect through standard monitoring. T-18 (Pipeline Privilege Escalation) completes this tier: a developer or attacker with repository write access can craft a malicious workflow step to extract production secrets or trigger unauthorised deployments, with potentially catastrophic impact on system confidentiality and integrity.

6.3.3 Medium Risks

T-02, T-08, T-09, T-11, and T-13 are mitigable with well-established, low-effort controls — rate limiting, CSRF tokens, branch protection rules, HTTPS enforcement, and audit logging. Their medium rating reflects residual risk in the absence of those controls, not inherent difficulty of mitigation. T-09 (Unauthorised Branch Push) warrants particular attention: although its likelihood is constrained by the requirement for valid repository write credentials, the impact of malicious code reaching production is high, and it is fully mitigated by enforcing branch protection and mandatory code review approvals. These threats should be addressed in the security requirements but are unlikely to require significant implementation effort.

6.3.4 Low Risks

T-01, T-17, and T-15 have a limited attack surface or bounded impact. T-01 (session hijacking) requires a precondition such as XSS or active network sniffing and is largely mitigated by HTTPS and correct cookie flags. T-17 (reviewer sharing bypass) has a narrow attack surface requiring prior knowledge of sharing relationships. T-15 (resource exhaustion) is bounded by per-user storage quotas and rate limiting when those controls are in place.

6.4 Threat Prioritisation Summary

Based on the risk assessment results, threats are grouped into four priority tiers for the requirements elicitation and prioritisation stages:

Priority	Threats	Primary Security Requirement Focus
Critical	T-04, T-05, T-07	Input validation, parameterised queries, file type enforcement, safe OS API usage
High	T-03, T-06, T-10, T-12, T-14, T-16, T-18	Server-side authorisation on all endpoints, role-based access control, output encoding, CSP headers, server-side ownership checks on write operations, least-privilege CI/CD permissions, pinned dependency versions with digest verification, automated supply chain scanning, secret management via dedicated vault or environment secrets
Medium	T-02, T-08, T-09, T-11, T-13	Login rate limiting, account lockout, CSRF token validation, SameSite and Secure cookie flags, HTTPS enforcement, branch protection rules and mandatory code review approvals, comprehensive audit logging for security-relevant actions
Low	T-01, T-15, T-17	HTTPS enforcement and correct cookie flags, sharing parameter validation, upload quotas and rate limiting

Table 6.2: Threat prioritisation summary for requirements elicitation.

7. Elicitation of Security Requirements

To elicit security requirements, we adopted the Accelerated Requirements Method (ARM), as suggested in the SQUARE methodology. This technique promotes structured stakeholder interaction and is particularly effective in identifying security needs in complex systems.

7.1 BON

The core of ARM is the BON approach: **Brainstorm, Organize, and Name**. This process was applied based on the previously developed artifacts, including threat models, misuse cases, and attack trees.

7.1.1 Step 1 — Brainstorm

In the brainstorming phase, stakeholders and the requirements engineering team collaboratively identified potential security concerns, threats, and protection needs for the system.

The brainstorming session focused on:

- Unauthorized access to documents
- Leakage of sensitive data
- Improper authentication and session management
- Abuse of document sharing functionality
- Malicious file uploads
- Privilege escalation by users or reviewers
- Insider threats (e.g., administrators misusing privileges)

This phase produced a raw, unstructured list of security concerns derived from identified threats and misuse scenarios.

7.1.2 Step 2 — Organize

The identified concerns were then organized into meaningful categories based on security objectives and system components.

The main categories defined were:

- Authentication and Session Management

- Authorization and Access Control
- Data Protection (Confidentiality and Integrity)
- Input Validation and File Handling
- Audit and Logging
- System Availability
- Configuration and Deployment

This organization step helped structure the requirements and ensured coverage of all relevant security domains.

7.1.3 Step 3 — Name

In the final BON phase, each organized concern was transformed into clear, testable, and verifiable security requirements.

The following is the initial set of elicited security requirements:

- **SR-01 — Authentication:** The system shall require users to authenticate using secure credentials before accessing any functionality.
- **SR-02 — Session Management:** The system shall enforce session timeouts after a period of inactivity and invalidate sessions upon logout.
- **SR-03 — Access Control:** The system shall enforce role-based access control (RBAC), ensuring that users can only access resources they are authorized to.
- **SR-04 — Document Authorization:** The system shall ensure that only document owners and explicitly authorized users can access shared documents.
- **SR-05 — Data Confidentiality:** All sensitive data shall be protected in transit using secure communication protocols (e.g., HTTPS).
- **SR-06 — Data Integrity:** The system shall ensure that uploaded documents cannot be altered without proper authorization.
- **SR-07 — File Upload Security:** The system shall restrict uploaded file types and scan files for malicious content.
- **SR-08 — Logging:** The system shall log all security-relevant events, including login attempts, access to documents, and administrative actions.
- **SR-09 — Auditability:** Logs shall be protected against tampering and be available for audit purposes.
- **SR-10 — Availability:** The system shall ensure availability by mitigating denial-of-service conditions where feasible.
- **SR-11 — Admin Controls:** Administrative actions (e.g., enabling/disabling users) shall require proper authorization and be logged.
- **SR-12 — Password Policy:** The system shall enforce strong password policies, including minimum length, complexity, and periodic renewal.

- **SR-13 — Account Lockout:** The system shall temporarily lock user accounts after a defined number of failed login attempts.
- **SR-14 — Secure Session Cookies:** Session identifiers shall be stored in secure, HTTP-only cookies.
- **SR-15 — Least Privilege:** The system shall enforce the principle of least privilege for all users and roles.
- **SR-16 — Privilege Escalation Prevention:** The system shall prevent unauthorized privilege escalation attempts.
- **SR-17 — Data Encryption at Rest:** Sensitive data and stored documents shall be encrypted at rest.
- **SR-18 — Secure File Storage:** Uploaded documents shall be stored outside publicly accessible directories.
- **SR-19 — Metadata Protection:** Document metadata shall be protected against unauthorized access and modification.
- **SR-20 — Injection Protection:** The system shall sanitize and validate all inputs to prevent SQL injection, command injection, and XSS attacks.
- **SR-21 — API Security:** All API endpoints shall enforce authentication and authorization checks.
- **SR-22 — Rate Limiting:** The system shall implement rate limiting to prevent brute-force and abuse attacks.
- **SR-23 — File Size Limits:** The system shall enforce maximum file size limits for uploads.
- **SR-24 — File Content Validation:** The system shall validate file content (not only extension) to prevent disguised malicious files.
- **SR-25 — Intrusion Detection:** The system shall detect and flag suspicious activities such as repeated failed logins or abnormal access patterns.
- **SR-26 — Log Retention:** Security logs shall be retained for a defined period to support forensic analysis.
- **SR-27 — Admin Action Verification:** Critical administrative actions shall require additional verification (e.g., re-authentication).
- **SR-28 — User Management Security:** Only administrators shall be able to enable or disable user accounts.
- **SR-29 — Backup and Recovery:** The system shall perform regular backups and support secure recovery of data.
- **SR-30 — Fault Tolerance:** The system shall handle failures gracefully without compromising security.
- **SR-31 — Secure Configuration:** Default configurations shall follow secure defaults and avoid unnecessary services.

- **SR-32 — Dependency Security:** The system shall use updated and secure third-party dependencies.
- **SR-33 — Data Minimization:** The system shall only collect and store necessary user data.
- **SR-34 — Access Transparency:** Users shall be able to view who has access to their shared documents.
- **SR-35 — CSRF Protection:** The system shall enforce Cross-Site Request Forgery protections on all state-changing endpoints, including CSRF token validation and SameSite cookie policy enforcement.
- **SR-36 — Pipeline Integrity:** The CI/CD pipeline shall enforce branch protection rules, require code review approvals, and prevent direct pushes to production branches.
- **SR-37 — Automated Security Scanning:** The CI/CD pipeline shall integrate automated security analysis tools (e.g., SAST, dependency scanning, secret detection) as mandatory pipeline stages that block deployment on failure.
- **SR-38 — Container Security:** Container images used in the pipeline shall be scanned for known vulnerabilities before deployment, and only images from trusted registries shall be permitted.
- **SR-39 — Secret Management:** Secrets, API keys, and credentials shall never be hardcoded in source code or CI/CD configuration files; a dedicated secret management mechanism shall be used.

7.1.4 Detailed Requirements Traceability Index

SR ID	Requirement Summary	Threats	Misuse Cases	Goals
SR-01	Secure user authentication required	T-01, T-02	MC-03, MC-05	G-01
SR-02	Session timeout and invalidation	T-01, T-13	MC-05	G-01, G-03
SR-03	Role-based access control enforcement	T-12, T-16	MC-01, MC-04	G-01, G-02
SR-04	Access restricted to authorised document users	T-12, T-17	MC-01	G-02, G-03
SR-05	Secure communication (HTTPS)	T-13	MC-05	G-03
SR-06	Prevent unauthorised data modification	T-03	MC-01	G-04
SR-07	Secure file upload restrictions	T-04	MC-02	G-05
SR-08	Logging of security-relevant events	T-11	—	G-08
SR-09	Log protection against tampering	T-11	—	G-04, G-08

SR ID	Requirement Summary	Threats	Misuse Cases	Goals
SR-10	Ensure system availability	T-15	MC-09	G-06
SR-11	Secure admin actions with logging	T-16	MC-04	G-01, G-08
SR-12	Enforce strong password policy	T-02	MC-03	G-01
SR-13	Account lockout after failed attempts	T-02	MC-03	G-01
SR-14	Secure session cookies (Secure, HttpOnly)	T-01, T-13	MC-05	G-01, G-03
SR-15	Enforce least privilege principle	T-16, T-17	MC-01, MC-04	G-02
SR-16	Prevent unauthorised privilege escalation	T-16	MC-04	G-01, G-02
SR-17	Encrypt sensitive data at rest	T-13	MC-05	G-03
SR-18	Secure file storage outside public dirs	T-04, T-07	MC-02, MC-08	G-03, G-04
SR-19	Protect document metadata	T-12	MC-01	G-03
SR-20	Prevent injection attacks (SQL, XSS, OS)	T-05, T-06, T-07	MC-06, MC-07, MC-08	G-04, G-05
SR-21	Secure and authenticated API endpoints	T-12, T-16	MC-01, MC-04	G-01, G-02
SR-22	Implement rate limiting	T-02, T-15	MC-03, MC-09	G-01, G-06
SR-23	Enforce file size limits for uploads	T-15	MC-09	G-06
SR-24	Validate file content (not only extension)	T-04	MC-02	G-05
SR-25	Detect and flag suspicious activity	T-02, T-12	MC-03, MC-01	G-08
SR-26	Maintain log retention for forensic use	T-11	—	G-08
SR-27	Re-authentication for critical admin actions	T-16	MC-04	G-01, G-08
SR-28	Restrict user management to administrators	T-16	MC-04	G-01, G-02
SR-29	Backup and recovery mechanisms	T-04, T-15	MC-09	G-04, G-06
SR-30	Fault tolerance and graceful failure	T-15	MC-09	G-06
SR-31	Secure default configurations	—	—	G-01, G-07
SR-32	Secure and updated dependency management	T-10	MC-13	G-07
SR-33	Data minimization	—	—	G-03

SR ID	Requirement Summary	Threats	Misuse Cases	Goals
SR-34	Transparency of document access	T-12, T-17	MC-01	G-02, G-03
SR-35	CSRF protection on state-changing endpoints	T-08	MC-10	G-04, G-08
SR-36	CI/CD pipeline integrity enforcement (branch protection, reviews)	T-09	MC-11	G-04, G-07
SR-37	Automated security scanning in CI/CD (SAST, dependencies, secrets)	T-10, T-14, T-18	MC-12, MC-13	G-05, G-07, G-08
SR-38	Container image security validation	T-10	MC-13	G-05, G-07
SR-39	Secure secret management (no hardcoded secrets)	T-14	MC-12	G-03, G-07

Table 7.1: Requirements traceability index mapping security requirements to threats, misuse cases, and security goals.

7.1.5 Discussion

The elicitation process, conducted using the ARM technique and its BON structure, resulted in a comprehensive and well-structured set of security requirements. These requirements systematically address the threats and misuse scenarios identified in previous steps of the SQUARE methodology, particularly those derived from threat modeling and risk assessment.

A key strength of this approach is the explicit traceability between threats, misuse cases, and security requirements. The Requirements Traceability Index (RTI) ensures that each requirement is justified by a concrete security concern, reducing the risk of missing critical threats or introducing unnecessary controls. This alignment also supports consistency across the development lifecycle, enabling clear reasoning about why each security mechanism is required.

Additionally, the requirements were defined to be clear, unambiguous, and testable. Wherever possible, they describe *what* the system must enforce rather than *how* it should be implemented, avoiding premature design decisions. This distinction is essential to maintain flexibility in subsequent architectural and implementation phases.

The resulting set of requirements provides balanced coverage across key security domains, including authentication, authorization, data protection, input validation, logging, and availability. This ensures that the system addresses confidentiality, integrity, availability, and accountability concerns in a holistic manner.

Finally, this structured elicitation establishes a strong foundation for the next stages of the project. The defined requirements can be directly mapped to architectural security mechanisms and later validated through implementation and DevSecOps practices, ensuring continuous enforcement and verification throughout the software development lifecycle.

8. Categorization and Prioritization of Requirements

Following the ARM methodology and aligned with the SQUARE Step 7 guidance, the elicited security requirements were grouped into meaningful categories based on their security objectives and the system components they protect. This categorization directly reflects the organizational structure established in the Brainstorm–Organize–Name (BON) process and serves as a preparatory step for requirements prioritization.

Group	Category	Requirements
Group A	Authentication and Session Management	SR-01 — Secure user authentication required SR-02 — Session timeout and invalidation SR-12 — Enforce strong password policy SR-13 — Account lockout after failed attempts SR-14 — Secure session cookies (Secure, HttpOnly) SR-35 — CSRF protection on state-changing endpoints
Group B	Authorization and Access Control	SR-03 — Role-based access control enforcement SR-04 — Access restricted to authorised document users SR-11 — Secure admin actions with logging SR-15 — Enforce least privilege principle SR-16 — Prevent unauthorised privilege escalation SR-21 — Secure and authenticated API endpoints SR-27 — Re-authentication for critical admin actions SR-28 — Restrict user management to administrators SR-34 — Transparency of document access
Group C	Data Protection (Confidentiality and Integrity)	SR-05 — Secure communication (HTTPS) SR-06 — Prevent unauthorised data modification SR-17 — Encrypt sensitive data at rest SR-18 — Secure file storage outside public directories SR-19 — Protect document metadata SR-33 — Data minimization

Group	Category	Requirements
Group D	Input Validation and File Handling	SR-07 — Secure file upload restrictions SR-20 — Prevent injection attacks (SQL, XSS, OS) SR-23 — Enforce file size limits for uploads SR-24 — Validate file content (not only extension)
Group E	Audit and Logging	SR-08 — Logging of security-relevant events SR-09 — Log protection against tampering SR-25 — Detect and flag suspicious activity SR-26 — Maintain log retention for forensic use
Group F	System Availability and Resilience	SR-10 — Ensure system availability SR-22 — Implement rate limiting SR-29 — Backup and recovery mechanisms SR-30 — Fault tolerance and graceful failure
Group G	Configuration and Deployment	SR-31 — Secure default configurations SR-32 — Secure and updated dependency management SR-36 — CI/CD pipeline integrity enforcement (branch protection, reviews) SR-37 — Automated security scanning in CI/CD (SAST, dependencies, secrets) SR-38 — Container image security validation SR-39 — Secure secret management (no hard-coded secrets)

Table 8.1: Grouped security requirements following the ARM categorization step.

8.1 Prioritize Requirements

Requirements were prioritized using the risk assessment results from Chapter 6 as the primary input, following the SQUARE Step 8 guidance. Each requirement was assigned a priority tier — **Critical**, **High**, **Medium**, or **Low** — consistent with the threat prioritization summary established in Table 6.2. Requirements addressing Critical and High risks must be implemented in the first development iteration; Medium and Low priorities are addressed subsequently or as resources allow.

Priority	Requirements	Rationale	Driven by
Critical	SR-07, SR-18, SR-20, SR-24	Injection and file upload cluster (SR-07, SR-18, SR-20, SR-24): address SQL injection, OS command injection, XSS, and malicious file upload (T-04, T-05, T-07) — the three highest-risk threats, exploitable by any authenticated user and leading to full server compromise or database takeover.	T-04, T-05, T-07 (Critical risk)

Priority	Requirements	Rationale	Driven by
High	SR-03, SR-04, SR-06, SR-11, SR-15, SR-16, SR-19, SR-21, SR-25, SR-27, SR-28, SR-34, SR-37, SR-38, SR-39	Address broken access control (T-12, T-16), unauthorised document tampering (T-03), XSS (T-06), pipeline privilege escalation (T-18), supply chain attacks (T-10), and secret leakage via pipeline (T-14). These threats are silent by default or exploit the absence of basic server-side controls, and all can escalate to full account or platform takeover. The CI/CD security cluster (SR-37, SR-38, SR-39) enforces automated security scanning, container image validation, and secret management, ensuring malicious or vulnerable code and exposed credentials cannot reach production undetected (T-10, T-14, T-18).	T-03, T-06, T-10, T-12, T-14, T-16, T-18 (High risk)
Medium	SR-01, SR-02, SR-05, SR-08, SR-09, SR-12, SR-13, SR-14, SR-17, SR-22, SR-26, SR-35, SR-36	Address brute force authentication (T-02), CSRF (T-08), sensitive data exposure (T-13), missing audit logs (T-11), and unauthorised branch push (T-09). SR-36 specifically enforces branch protection rules and mandatory code review approvals, fully mitigating T-09 at low implementation cost. All remaining requirements are addressed by well-established, low-effort controls — rate limiting, cookie flags, CSRF tokens, HTTPS enforcement, and audit logging — and carry medium residual risk in their absence.	T-02, T-08, T-09, T-11, T-13 (Medium risk)
Low	SR-10, SR-23, SR-29, SR-30, SR-31, SR-32, SR-33	Address session hijacking (T-01), resource exhaustion (T-15), and reviewer sharing bypass (T-17). These have a limited attack surface or bounded impact, and their mitigations (HTTPS, cookie flags, upload quotas, sharing parameter validation) are low-cost to implement.	T-01, T-15, T-17 (Low risk)

Table 8.2: Prioritized security requirements derived from the risk assessment results.

8.2 Discussion

The categorisation step consolidated the 39 security requirements into seven coherent groups — Authentication and Session Management (A), Authorization and Access Control (B), Data Protection (C), Input Validation and File Handling (D), Audit and Logging (E), System Availability and Resilience (F), and Configuration and Deployment (G) — mirroring the security domains identified during the BON brainstorming phase. This structure facilitates traceability from high-level goals to concrete implementation tasks and ensures that each system component is explicitly covered by at least one requirement group.

The prioritisation step reinforces the conclusions of the risk assessment in Chapter 6. The Critical tier is dominated by Group D: SR-07, SR-18, SR-20, and SR-24 form a single injection and file upload cluster that addresses SQL injection, OS command injection, XSS, and malicious file upload (T-04, T-05, T-07). These three threats share a common root cause — user-supplied input incorporated into sensitive operations without validation — are exploitable by any authenticated user, and can lead to full server compromise or database takeover. Implementation of all Critical requirements must precede all other security work.

The High tier concentrates primarily on Groups B and C, reflecting the fact that broken access control and weak data integrity enforcement are the most prevalent and impactful vulnerability classes in public-facing web applications. SR-03, SR-04, SR-15, SR-16, SR-21, SR-27, and SR-28 address the absence of server-side role and ownership checks (T-12, T-16, T-03), while SR-06 and SR-19 reinforce data and metadata integrity. SR-11 and SR-34 ensure that administrative actions are both controlled and transparent. SR-25 (Group E) is also High-priority, as detecting anomalous access patterns is essential to surface the silent threats in this tier. The CI/CD security cluster — SR-37, SR-38, and SR-39 (Group G) — completes the High tier: automated security scanning, container image validation, and secret management collectively prevent malicious or vulnerable code and exposed credentials from reaching production (T-10, T-14, T-18), covering attack vectors that are largely invisible to standard application-level monitoring.

The Medium tier is dominated by transport-layer, session-management, and audit controls from Groups A and E. SR-01, SR-02, SR-12, SR-13, and SR-14 harden authentication and session handling; SR-05 and SR-17 enforce HTTPS and encryption at rest; SR-22 introduces rate limiting; SR-35 adds CSRF token validation; and SR-08, SR-09, and SR-26 establish tamper-resistant audit logging. SR-36 (Group G) is also Medium-priority: it enforces branch protection rules and mandatory code review approvals, fully mitigating the unauthorised branch push threat (T-09) at low implementation cost. All controls in this tier are well understood and low-cost to introduce, but carry non-negligible residual risk if absent.

The Low tier captures availability, configuration, and privacy requirements from Groups F and G, together with SR-33 (data minimisation). SR-10, SR-23, SR-29, and SR-30 bound the impact of resource exhaustion and storage failures; SR-31 and SR-32 enforce secure defaults and dependency hygiene. These requirements are essential for a complete security posture but do not individually represent immediate exploitation risk when higher-priority controls are already in place.

This prioritised and categorised set of requirements provides a clear, risk-driven implementation roadmap for the secure architecture and implementation stages that follow.

9. Secure Architecture and Design

This chapter translates the security requirements and threat analysis developed in the previous stages into concrete architectural decisions. Rather than describing the full system architecture, the focus is on security-relevant mechanisms, trust boundaries, and enforcement points that directly address the threats and requirements identified in Chapters 3 through 7.

9.1 Trust Boundaries and Security Domains

A trust boundary separates two areas with different levels of trust, where data crossing the boundary must be validated, authenticated, or otherwise controlled. Four principal trust boundaries were identified in the document management platform.

9.1.1 Identified Trust Boundaries

Boundary	Separates	Attack Surface
TB-01 — Internet / Application	Public internet (unauthenticated clients) and the application server.	Brute force (T-02), injection (T-05, T-06, T-07), malicious uploads (T-04), CSRF (T-08), DoS (T-15). All user-supplied input crosses here.
TB-02 — Application / Database	Application server and the PostgreSQL database instance.	SQL injection (T-05), unauthorised data access if application credentials are overly broad; a compromised application server must not grant direct database shell access.
TB-03 — Application / File Storage	Application server and the document file-storage volume.	Path traversal, malicious file execution (T-04, T-07); files must not be served directly from the storage volume by a web server process.
TB-04 — Developer / CI/CD / Production	Developer workstations and the source repository, and between the CI/CD pipeline and the production environment.	Unauthorised branch push (T-09), supply chain attacks (T-10), secret leakage (T-14), pipeline privilege escalation (T-18).

Table 9.1: Trust boundaries and their attack surfaces.

9.1.2 Security Domains

The system is partitioned into three security domains that follow the trust boundaries above.

- **Public Domain** — the browser-based client running on the user’s machine. This domain is fully untrusted: any data, token, or claim originating here must be validated server-side before acting on it.
- **Application Domain** — the containerized application server. This is the primary enforcement layer: authentication, authorization, input validation, session management, and audit logging are all enforced here. Components within this domain communicate over a private network not exposed to the public.
- **Data Domain** — the PostgreSQL database and the document storage volume. Both components are isolated from the public internet and accessible only from the Application Domain using dedicated, least-privilege credentials.

9.2 Security Enforcement Points

This section describes where and how each key security mechanism is enforced in the architecture, and maps each enforcement point back to the threats it mitigates and the requirements it satisfies.

9.2.1 EP-01 — HTTPS Termination and Transport Security

Location: Trust boundary TB-01, before any application logic.

All communication between the client and the application server is encrypted using TLS. The application must redirect all plain HTTP requests to HTTPS and must set the **Strict-Transport-Security** (HSTS) response header to prevent protocol downgrade attacks. Session cookies must carry the **Secure** attribute so they are never transmitted over unencrypted connections.

Threats mitigated: T-01 (session hijacking via sniffing), T-13 (sensitive data exposure in transit). *Requirements addressed:* SR-05, SR-14.

9.2.2 EP-02 — Authentication and Session Management

Location: Application Domain — authentication middleware, executed before any protected route handler.

All access to platform functionality requires a valid, server-issued session token. The authentication flow enforces the following:

- Passwords are stored as salted hashes using a modern adaptive algorithm (e.g., bcrypt or Argon2); plaintext passwords are never stored or logged.
- Session tokens are generated using a cryptographically secure random source, stored server-side, and bound to a session record that carries the authenticated user’s identity and role.
- Session cookies are issued with **Secure**, **HttpOnly**, and **SameSite=Strict** attributes to prevent JavaScript access and cross-site transmission.

- Sessions expire after a configurable period of inactivity and are invalidated server-side on logout, ensuring that tokens cannot be replayed after the session ends.

Threats mitigated: T-01 (session hijacking), T-02 (brute force), T-13 (credential exposure).

Requirements addressed: SR-01, SR-02, SR-12, SR-13, SR-14.

9.2.3 EP-03 — Rate Limiting and Account Lockout

Location: Trust boundary TB-01 — applied to the login and upload endpoints before reaching authentication or business logic.

The login endpoint is protected by a per-IP and per-account rate limiter. After a configurable number of consecutive failed authentication attempts, the targeted account is temporarily locked and the event is logged and alerted. The upload endpoint is similarly rate-limited to bound the impact of resource-exhaustion abuse.

Threats mitigated: T-02 (brute force), T-15 (resource exhaustion via abusive uploads).

Requirements addressed: SR-13, SR-22, SR-23.

9.2.4 EP-04 — Role-Based Access Control (RBAC)

Location: Application Domain — authorization middleware, applied server-side on every protected route handler, after authentication.

Every request to a protected endpoint is subject to a server-side authorization check that verifies the authenticated user's role and, where applicable, their ownership of or explicit share permission to the target resource. Client-supplied role claims or resource identifiers are never trusted without verification.

The three roles defined in the system (User, Reviewer, Administrator) are enforced at the middleware layer:

- **Administrator-only endpoints** (user listing, account enable/disable) are decorated with a role guard that rejects requests from non-administrators with HTTP 403.
- **Document access endpoints** verify that the requesting user is either the document owner or an explicitly named reviewer before serving the resource.
- **Reviewer endpoints** verify that the document was explicitly shared with the requesting user; possession of a valid session is insufficient.

Threats mitigated: T-03 (IDOR tampering), T-12 (IDOR disclosure), T-16 (privilege escalation via API), T-17 (reviewer sharing bypass). *Requirements addressed:* SR-03, SR-04, SR-11, SR-15, SR-16, SR-21, SR-27, SR-28, SR-34.

9.2.5 EP-05 — Input Validation and Injection Prevention

Location: Application Domain — at every point where user-supplied data enters business logic or is passed to an external system (database, OS, browser).

This enforcement point addresses the three critical injection threats identified in the risk assessment:

- **SQL Injection (T-05):** All database interactions use parameterized queries or ORM abstractions that prevent user input from being interpreted as SQL. Raw string concatenation into query strings is prohibited. The database user account used by the application is scoped to the minimum required permissions (SELECT, INSERT, UPDATE, DELETE on specific tables; no DDL access).
- **OS Command Injection (T-07):** No user-supplied input (e.g., filenames) is passed to OS-level commands. File operations are performed exclusively through safe Python filesystem APIs (`pathlib`, `shutil`). Where subprocess calls are unavoidable, they use `shell=False` with a pre-validated argument list.
- **Cross-Site Scripting (T-06):** All user-supplied data rendered in HTML responses is HTML-encoded using the template engine’s auto-escaping mechanism. A Content-Security-Policy (CSP) header is enforced on all responses to restrict executable script sources.

Threats mitigated: T-05 (SQL injection), T-06 (XSS), T-07 (OS command injection). *Requirements addressed:* SR-20.

9.2.6 EP-06 — File Upload Validation and Secure Storage

Location: Application Domain — upload handler; Data Domain — document storage volume.

Uploaded files are subject to multi-layer validation before being persisted:

- The declared MIME type and the file extension are checked against an allowlist of permitted document types.
- The actual file content (magic bytes) is inspected independently of the client-supplied MIME type and filename to detect disguised executables or web shells.
- File size is enforced against a configurable maximum before the upload is accepted.
- Accepted files are stored in a directory that is not directly served by any web server process. Downloads are served by the application server only after authorization has been verified, using a server-side read and stream approach rather than direct filesystem URLs.
- Stored filenames are replaced by server-generated identifiers (e.g., UUIDs) to prevent path traversal and filename-based injection.

Threats mitigated: T-04 (malicious file upload), T-07 (OS command injection via filename), T-15 (resource exhaustion). *Requirements addressed:* SR-07, SR-18, SR-23, SR-24.

9.2.7 EP-07 — CSRF Protection

Location: Application Domain — applied to all state-changing endpoints (POST, PUT, DELETE).

A synchronizer token pattern is implemented: the server issues a unique, unpredictable CSRF token per session, embeds it in every HTML form as a hidden field, and validates its presence and correctness on every state-changing request. In addition, session cookies carry `SameSite=Strict`, providing defence-in-depth against cross-origin request forgery even in browsers that do not support CSRF tokens natively.

Threats mitigated: T-08 (CSRF). *Requirements addressed:* SR-35.

9.2.8 EP-08 — Audit Logging

Location: Application Domain — a dedicated audit logging component that is invoked by every security-relevant route handler.

All security-relevant events are recorded with sufficient context for forensic analysis: successful and failed authentication attempts, document upload, download, deletion and sharing events, administrative actions, and authorization failures. Log records are structured (e.g., JSON), include a timestamp, user identifier, action type, target resource, and outcome, and are written to a storage location that is not writable by the application process to prevent tampering.

Threats mitigated: T-11 (repudiation / missing audit logs). *Requirements addressed:* SR-08, SR-09, SR-25, SR-26.

9.2.9 EP-09 — CI/CD Security Controls

Location: Trust boundary TB-04 — source code repository and CI/CD pipeline.

Security controls are embedded into the software delivery pipeline to enforce the requirements of Group G at every commit and merge:

- **Branch protection:** Direct pushes to the `main` and `production` branches are blocked for all identities, including repository owners. At least one peer-review approval is required before any merge. This mitigates T-09 (unauthorised branch push).
- **SAST:** A static application security testing tool (e.g., Bandit for Python) is integrated as a mandatory, blocking pipeline stage. Findings above a configurable severity threshold cause the pipeline to fail.
- **Dependency and supply chain scanning:** Software Composition Analysis (SCA) runs on every build to detect known CVEs in used packages. Dependencies are pinned to specific versions with cryptographic hash verification. This mitigates T-10 (supply chain attack).
- **Secret detection:** A secret-scanning tool (e.g., Gitleaks or `detect-secrets`) runs on every commit and pull request. Detection of a credential pattern causes the pipeline to fail immediately and triggers an alert. This mitigates T-14 (secret leakage via pipeline).
- **Container image scanning:** Base images are pulled from a trusted registry and verified by digest before use. An image vulnerability scanner (e.g., Trivy) runs as a mandatory stage and blocks deployment if critical CVEs are present.
- **Least-privilege pipeline credentials:** CI/CD job tokens and service accounts are scoped to the minimum required permissions per environment. Production deployments are restricted to a dedicated protected workflow requiring manual approval. This mitigates T-18 (pipeline privilege escalation).
- **Secret management:** All secrets, API keys, and credentials are injected into pipeline jobs exclusively through the CI/CD platform's encrypted secret store. Hard-coded secrets in source code or configuration files are prohibited and detected by the secret-scanning stage described above.

Threats mitigated: T-09, T-10, T-14, T-18. *Requirements addressed:* SR-36, SR-37, SR-38, SR-39.

9.2.10 EP-10 — Encryption at Rest

Location: Data Domain — document storage volume and database.

Sensitive data persisted by the platform must be protected against unauthorised disclosure in the event of physical or logical access to the underlying storage. The following controls are applied:

- **Database encryption:** The PostgreSQL instance is deployed on an encrypted storage volume. Sensitive fields (e.g., password hashes, session records) are never stored in plaintext; passwords are stored exclusively as salted hashes produced by an adaptive algorithm (bcrypt or Argon2).
- **Document storage encryption:** The file storage volume hosting uploaded documents is encrypted at the filesystem or block-device level, ensuring that raw access to the volume does not yield readable document content.
- **Key management:** Encryption keys are never stored alongside the data they protect. Keys are managed through the deployment environment’s secret management mechanism (see EP-09) and rotated on a defined schedule.

Threats mitigated: T-13 (sensitive data exposure via insecure storage). *Requirements addressed:* SR-17.

9.2.11 EP-11 — Operational Resilience and Secure Defaults

Location: Application Domain and deployment configuration — applied at deployment time and enforced through infrastructure configuration.

This enforcement point groups three closely related operational controls that share the same deployment-time scope and collectively ensure the platform remains secure and recoverable under failure conditions.

- **Secure default configuration (SR-31):** The deployment configuration applies secure defaults for all components. Unnecessary services, ports, and features are disabled. Debug modes, verbose error pages, and default credentials are prohibited in all non-development environments. Security response headers (**Content-Security-Policy**, **X-Content-Type-Options**, **X-Frame-Options**, **Referrer-Policy**) are applied globally by middleware rather than per-route, so that new routes inherit them automatically.
- **Backup and recovery (SR-29):** Automated backups of the database and document storage are performed on a defined schedule and stored in a location that is isolated from the production environment. Backup integrity is verified periodically through test restores. Backup storage is subject to the same access controls and encryption requirements as primary data (EP-10).
- **Fault tolerance and graceful failure (SR-30):** The application handles unexpected errors without exposing internal state, stack traces, or configuration details to the client. All unhandled exceptions are caught by a global error handler that returns a generic error response and records the full error detail to the internal audit log (EP-08). Service failures in one component (e.g., the database becoming temporarily unavailable) are handled gracefully and do not result in a security degradation such as bypassing authentication or authorization checks.

Threats mitigated: T-15 (resource exhaustion and storage failure). *Requirements addressed:* SR-29, SR-30, SR-31.

9.3 Architectural Overview

Figure 9.1 presents a high-level view of the security architecture, organised into four distinct security domains separated by explicit trust boundaries.

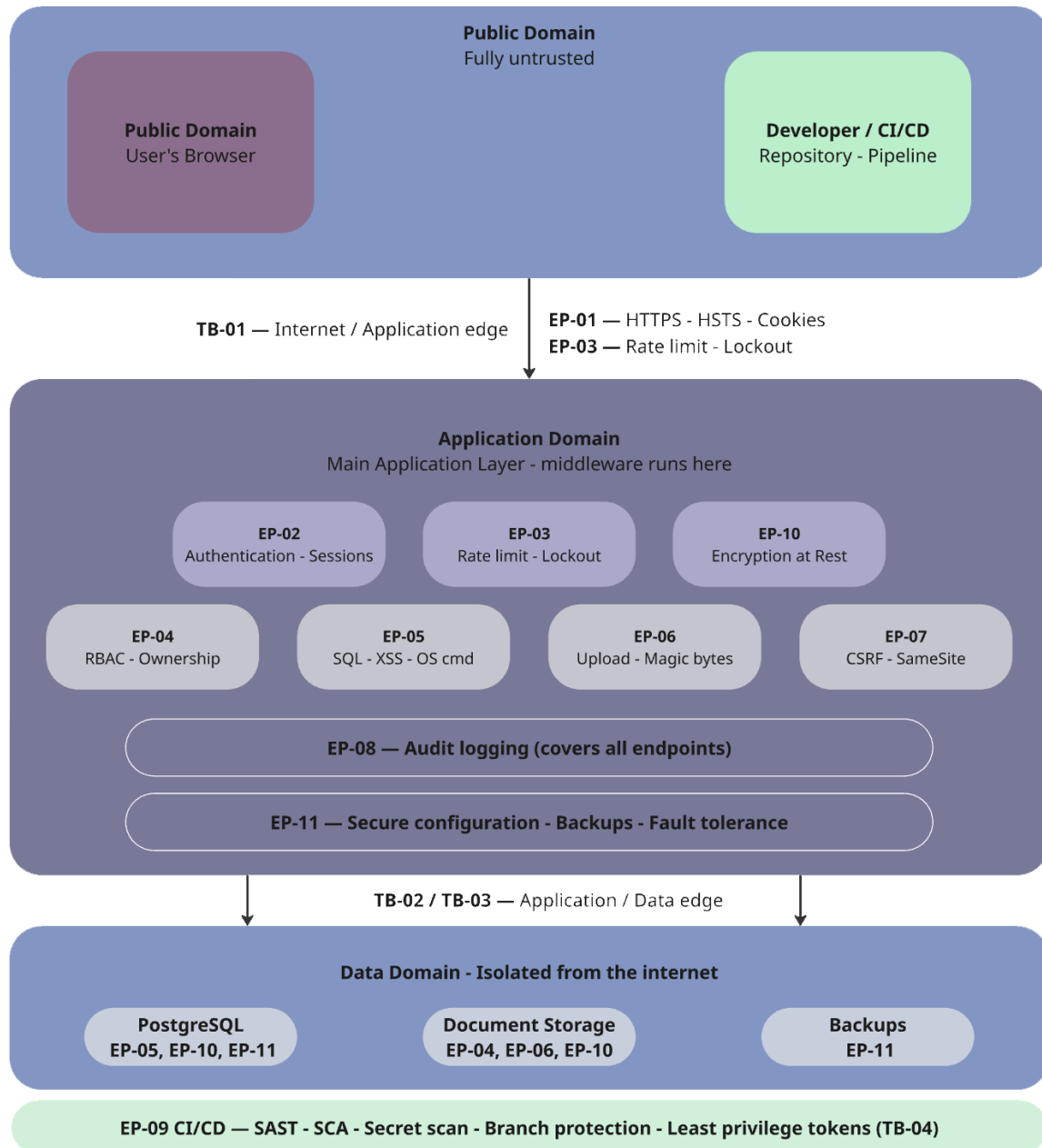


Figure 9.1: Security architecture overview — domains, trust boundaries, and enforcement point placement.

The enforcement points are distributed across the Application Domain so that every request crossing TB-01 is processed through layered controls before reaching the Data Domain.

EP-08 (audit logging) and EP-11 (secure configuration and fault tolerance) span the entire Application Domain as cross-cutting concerns, ensuring that all security-relevant events are recorded and that the system operates securely by default regardless of the specific endpoint being invoked.

9.4 Architectural Security Principles

This section explains how the proposed architecture applies the four security principles required by the project specification.

9.4.1 Least Privilege

Least privilege is applied at every layer of the architecture:

- **Application layer:** Each user role (User, Reviewer, Administrator) has access only to the endpoints and resources explicitly assigned to that role (EP-04). No role implicitly inherits permissions from a higher-privileged role.
- **Database layer:** The application connects to the database with a dedicated account that holds only the DML permissions required for normal operation. DDL access, superuser rights, and access to tables outside the application schema are denied.
- **CI/CD layer:** Pipeline job tokens are scoped per environment (staging vs. production). Production secrets are accessible only to the deployment job, not to build or test jobs (EP-09).
- **File storage:** The application process reads and writes the document storage directory; the web server process has no direct access to that directory, preventing direct file serving that could bypass authorization.

Supporting requirements: SR-15, SR-16, SR-28, SR-39.

9.4.2 Defense in Depth

No single security control is relied upon as the sole protection against any threat. Multiple, independent enforcement points are layered so that the failure of one does not result in a successful attack:

- **Against injection (T-05, T-06, T-07):** Parameterized queries prevent SQL injection even if input validation is bypassed; auto-escaping prevents XSS even if CSP is misconfigured; safe file APIs prevent OS command injection even if filename validation fails (EP-05).
- **Against malicious uploads (T-04):** Extension allowlisting, MIME type inspection, and magic byte validation are independent checks; sandboxed storage prevents execution even if all three fail (EP-06).
- **Against session attacks (T-01, T-08):** HTTPS encryption, HttpOnly and Secure cookie flags, SameSite policy, and CSRF tokens each independently reduce the attack surface (EP-01, EP-02, EP-07).
- **Against pipeline threats (T-09, T-10, T-14, T-18):** Branch protection, SAST, SCA, secret scanning, and container scanning are independent blocking stages (EP-09).

9.4.3 Secure Defaults

The system is configured with secure defaults that require explicit opt-out rather than opt-in:

- All routes are protected by authentication by default; public routes (e.g., the login page) are explicitly whitelisted. A developer omitting an authorization decorator on a new endpoint will cause a denied request, not an open one.
- Template auto-escaping is enabled globally; a developer must explicitly mark output as safe to disable it.
- HTTPS redirection and HSTS are enabled by default in the deployment configuration.
- The `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, and `Referrer-Policy` headers are set on all responses by a middleware applied globally, not per-route.
- The database connection uses parameterized queries by default through the ORM; raw query execution requires an explicit bypass that is flagged by the SAST tool.

Supporting requirements: SR-31.

9.4.4 Separation of Concerns

Security responsibilities are clearly separated between architectural components so that each component has a single, well-defined security role:

- **Authentication** is handled exclusively by the authentication middleware (EP-02); business logic handlers do not perform authentication checks.
- **Authorization** is handled exclusively by the RBAC middleware (EP-04); business logic does not contain role checks or ownership verification logic.
- **Input validation** is performed at the boundary where data enters the application (EP-05, EP-06), before it reaches any business logic handler.
- **Audit logging** is performed by a dedicated logging component (EP-08); route handlers call a logging API rather than writing log records directly.
- **File serving** is performed exclusively by the application server after authorization; the web server or storage volume does not serve files directly.

This separation ensures that each security mechanism can be independently reviewed, tested, and updated without risk of inadvertently disabling another. It also supports the defense-in-depth principle: because concerns are separated, a vulnerability in one component does not automatically compromise the security logic of another.

9.5 Traceability to Security Requirements

Table 9.2 summarises the mapping between the architectural enforcement points defined in this chapter and the security requirements elicited in Chapter 7.

EP	Mechanism	Requirements	Threats
EP-01	HTTPS termination, HSTS, Secure cookies	SR-05, SR-14	T-01, T-13
EP-02	Authentication, session management, cookie flags	SR-01, SR-02, SR-12, SR-13, SR-14	T-01, T-02, T-13
EP-03	Rate limiting, account lockout	SR-10, SR-13, SR-22, SR-23	T-02, T-15
EP-04	RBAC, ownership checks, role guards	SR-03, SR-04, SR-06, SR-11, SR-15, SR-16, SR-19, SR-21, SR-27, SR-28, SR-34	T-03, T-12, T-16, T-17
EP-05	Parameterized queries, safe APIs, auto-escaping, CSP	SR-20	T-05, T-06, T-07
EP-06	File type validation, magic byte inspection, sandboxed storage, UUID naming	SR-07, SR-18, SR-23, SR-24	T-04, T-07, T-15
EP-07	CSRF token validation, SameSite cookies	SR-35	T-08
EP-08	Structured audit logging, tamper-resistant log storage	SR-08, SR-09, SR-25, SR-26	T-11
EP-09	Branch protection, SAST, SCA, secret scanning, container scanning, least-privilege tokens, secret management	SR-32, SR-36, SR-37, SR-38, SR-39	T-09, T-10, T-14, T-18
EP-10	Encryption at rest for sensitive data and stored documents (e.g., encrypted storage volumes or application-level encryption)	SR-17	T-13
EP-11	Operational resilience: secure default configurations, automated backups, graceful failure handling, and fault-tolerant deployment	SR-29, SR-30, SR-31	T-15

Table 9.2: Traceability from architectural enforcement points to security requirements and threats.

Note on SR-33 (Data Minimisation). SR-33 does not map to a specific architectural enforcement point because it is a design-time and policy-level requirement rather than a runtime mechanism. Compliance is achieved by ensuring that only necessary fields are

collected in data models and forms, and is verified through design review rather than through a deployable control.

Bibliography

- [1] N. R. Mead, E. D. Hough, and T. R. Stehney II, *Security Quality Requirements Engineering (SQUARE) Methodology*, Technical Report CMU/SEI-2005-TR-009, Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, Nov. 2005.

Disclaimer: Generative AI tools were used to support the writing process, particularly for language refinement, text organization, and clarity improvement. In addition, lecture slides provided during the course were used as supporting academic material.